

Iteracja i tablice

1. Program, co policzy iteracyjnie hipersilnię liczby n : $H(n) := 1^1 \cdot 2^2 \cdot \dots \cdot n^n$ (użytkownik podaje $n \in \mathbb{N}$, program liczy $H(n) \in \mathbb{N}$ (pseudokod + schemat).

2. Program, który policzy iteracyjnie przybliżenie wartości $\ln(1+x)$ rzędu n za pomocą szeregów Taylora (schemat LUB pseudokod): $\ln_n(1+x) = \sum_{i=1}^n \frac{(-1)^{i-1}}{i} x^i = \frac{1}{1}x - \frac{1}{2}x^2 + \frac{1}{3}x^3 - \frac{1}{4}x^4 + \dots + \frac{(-1)^{n-1}}{n}x^n$

1. Wejście: $x \in \mathbb{R}$ oraz rząd przybliżenia: $n \in \mathbb{N}$

2. Wyjście: wartość $\ln_n(1+x) \in \mathbb{R}$

3. Program, który będzie liczył to co program nr 2, ale do momentu, do którego poprawa kolejnych przybliżeń stanie się mniejsza niż określona dokładność ϵ podana przez użytkownika (pseudokod ORAZ schemat):

1. Wejście: $x \in \mathbb{R}$ oraz założona dokładność $\epsilon \in \mathbb{R}^+$.

2. Wyjście: przybliżenie $\ln(1+x)$ z dokładnością ϵ .

■ **Przykładowo:** program skończy na przybliżeniu rzędu $\ln_n(1+x)$ wg. wzoru z poprzedniego punktu, jeśli:

■ $|\ln_{n+1}(1+x) - \ln_n(1+x)| < \epsilon$.

■ To zadanie naturalnie zrobić za pomocą wariantu pętli `while`.

4. Program, który znajdzie maksymalny element w wejściowej, 1 wymiarowej tablicy liczb całkowitych o długości 100, $A \in \mathbb{N}^{100}$ (pseudokod LUB schemat).

1. jak zmodyfikować program aby zamiast tego znaleźć `arg max`, tj. pozycję maksimum w tablicy (jego *indeks*)?

5. Program, który znajdzie minimalny element w wejściowej, 2 wymiarowej tablicy liczb całkowitych o wymiarze 10×30 , $B \in \mathbb{N}^{10,30}$ (pseudokod ORAZ schemat)

1. Podobnie, jak zmodyfikować program, aby zamiast tego znaleźć `arg min` - pozycję minimum w dwuwymiarowej tablicy? Ile wartości trzeba zwrócić?(program może zwracać tabelę)

6. Mamy $A \in \mathbb{R}^{m,n}$ oraz odpowiadającą mu tablicę 1 wymiarową $B \in \mathbb{R}^{m \cdot n}$, która przechowuje te same dane w porządku "row-major". Użytkownik daje i, j . Napisać wzór na indeks wartości $A[i][j]$ w tablicy B , tj. na takie k , że $A[i][j] = B[k]$. Wzór może korzystać

z wartości i, j, m, n , dodawania i mnożenia. **Pamiętamy:** indeksowanie zaczynamy od zera, jak w C:

- o Przykładowo, jeśli $A \in \mathbb{R}^{3,4}$ to:

```
1  2  3  4
5  6  7  8
9 10 11 12
```

- to $B \in \mathbb{R}^{12}$, będzie wyglądać tak:

```
1 2 3 4 5 6 7 8 9 10 11 12
```

- pytamy, jak np. policzyć pozycję elementu np. $A[1, 3] = 7$ w tabeli B , na podstawie jego indeksów i, j w tabeli A , tutaj równych 1 i 3 oraz wymiarów m, n , tutaj równych 3,4.

7. Jak w 6, ale odpowiadająca tablica 1-wymiarowa $B \in \mathbb{R}^{m \cdot n}$ jest uporządkowana w porządku column-major. Dla podanego przykładu:

```
1 5 9 2 6 10 2 7 11 4 8 12
```

8. Zaproponować 3-wymiarową tablicę $A \in \mathbb{N}^{n,m,k}$ do przechowywania obrazów w formacie RGB o wymiarze `1000x700` pikseli.
9. Zaproponować 4-wymiarową tablicę A , która przechowuje 100 obrazów z punktu 8.
10. Napisać program, który dla wejściowego obrazu RGB o wymiarze `1000x700` przechowywanego w postaci 3-wymiarowej tablicy A jak wcześniej, policzy trzy wartości: $\bar{r}, \bar{g}, \bar{b}$ - średnie wartości nasycenia każdego z 3 kanałów `R,G,B` w wejściowym obrazie. Średnią liczymy po wszystkich pikselach w obrazie.
11. Napisać program, który dla wejściowych 100 obrazów o wymiarze `1000x700` w formacie R,G,B, przechowywanych jako 4-wymiarowa tablica, policzy 3 wymiarową tablicę $\bar{A} \in \mathbb{N}^{1000,700,3}$, która będzie przechowywała średnie wartości nasycenia każdego z kanałów `R,G,B` we **wszystkich** wejściowych obrazach, **na każdym z pikselów z osobna** (średnią liczymy po obrazach, inne wymiary zostają tak samo)